



PRACTICAL_GUIDE



HelixCode Practical Guide

OVERVIEW



This practical guide provides hands-on tutorials and code examples for using HelixCode's memory system, vector providers, and Cognition integration. Each tutorial includes complete, working code that you can run immediately.

WHAT YOU'LL LEARN

- Set up and configure HelixCode memory system
- Use different vector database providers
- Integrate with Cognition for advanced memory operations
- Implement conversation context management
- Build complete AI applications with memory
- Monitor and optimize performance



GETTING STARTED

Prerequisites

Go 1.19+

```
go version
```

Docker & Docker Compose

```
docker --version
```

```
docker-compose --version
```

Git

```
git --version
```

Python (for some vector databases)

```
python3 --version
```

Quick Setup

Clone repository

```
git clone https://github.com/helixcode/helixcode.git
```

```
cd helixcode
```

```
# Install dependencies
```

```
go mod tidy
```

```
# Start infrastructure
```

```
docker-compose -f docker/docker-compose.yml up -d
```

```
# Create configuration
```

```
cp helix.template.json helix.json
```

```
# Run example
```

```
go run examples/basic/main.go
```

TUTORIAL 1: BASIC MEMORY SYSTEM

LEARNING OBJECTIVES

- Initialize memory system
- Store and retrieve vectors
- Perform similarity search
- Manage collections

Complete Code Example

```
// examples/basic/main.go
```

```
package main
```

```
import (
```

```
    "context"
```

```
    "fmt"
```

```
    "log"
```

```
    "time"
```

```
    "dev.helix.code/internal/config"
```

```
    "dev.helix.code/internal/logging"
```

```
    "dev.helix.code/internal/memory"
```

```
    "dev.helix.code/internal/memory/providers"
```

```
)
```

```
func main() {
```

```
    // Create context
```

```
    ctx := context.Background()
```

```

// Initialize logger
logger := logging.NewLogger("basic-example")

// Load configuration
cfg, err := config.LoadFromFile("helix.json")
if err != nil {
    log.Fatal("Failed to load config:", err)
}

// Create vector provider manager
providerManager := providers.NewVectorProviderManager(
    cfg.Memory.VectorProviders,
    logger,
)

// Initialize providers
if err := providerManager.Initialize(ctx); err != nil {
    log.Fatal("Failed to initialize providers:", err)
}
defer providerManager.Shutdown(ctx)

// Create memory manager
memoryManager := memory.NewManager(
    providerManager,
    logger,
)

// Initialize memory
if err := memoryManager.Initialize(ctx, cfg); err != nil {
    log.Fatal("Failed to initialize memory:", err)
}
defer memoryManager.Shutdown(ctx)

// Run basic operations
if err := runBasicOperations(ctx, memoryManager); err != nil {
    {
        log.Fatal("Basic operations failed:", err)
    }
}

func runBasicOperations(ctx context.Context, manager
    *memory.Manager) error {
    fmt.Println("=== HelixCode Basic Memory Operations ===\n")

```

```

// 1. Create collection
fmt.Println("1. Creating collection...")
collectionConfig := &memory.CollectionConfig{
    Name:      "tutorial_collection",
    Dimension: 1536,
    Metric:    "cosine",
    Metadata: map[string]interface{}{
        "description": "Tutorial collection for basic
operations",
        "created_by":  "basic-tutorial",
    },
}

if err := manager.CreateCollection(ctx,
    "tutorial_collection", collectionConfig); err != nil {
    return fmt.Errorf("failed to create collection: %w", err)
}
fmt.Println("    Collection created successfully\n")

// 2. Store sample vectors
fmt.Println("2. Storing sample vectors...")
vectors := createSampleVectors()

if err := manager.Store(ctx, vectors); err != nil {
    return fmt.Errorf("failed to store vectors: %w", err)
}
fmt.Printf("    Stored %d vectors successfully\n\n",
    len(vectors))

// 3. Retrieve vectors
fmt.Println("3. Retrieving vectors...")
ids := []string{"doc1", "doc2", "doc3"}

retrieved, err := manager.Retrieve(ctx, ids)
if err != nil {
    return fmt.Errorf("failed to retrieve vectors: %w", err)
}

fmt.Printf("    Retrieved %d vectors:\n", len(retrieved))
for i, vec := range retrieved {
    fmt.Printf("    %d. ID: %s, Title: %v\n", i+1, vec.ID,
        vec.Metadata["title"])
}
fmt.Println()

```

```

// 4. Perform similarity search
fmt.Println("4. Performing similarity search...")
queryVector := createQueryVector()

query := &memory.VectorQuery{
    Vector:      queryVector,
    Collection:  "tutorial_collection",
    TopK:        3,
    Threshold:   0.7,
    Filters:     map[string]interface{}{
        "category": "technology",
    },
}

results, err := manager.Search(ctx, query)
if err != nil {
    return fmt.Errorf("failed to search: %w", err)
}

fmt.Printf("    Found %d similar documents:\n",
    len(results.Results))
for i, result := range results.Results {
    fmt.Printf("    %d. ID: %s, Score: %.4f\n", i+1,
        result.ID, result.Score)
    fmt.Printf("        Title: %v\n", result.Metadata["title"])
    fmt.Printf("        Category: %v\n",
        result.Metadata["category"])
}
fmt.Println()

// 5. Update vector metadata
fmt.Println("5. Updating vector metadata...")
metadata := map[string]interface{}{
    "last_updated": time.Now(),
    "updated_by":   "basic-tutorial",
    "priority":     "high",
}

if err := manager.UpdateMetadata(ctx, "doc1", metadata);
    err != nil {
    return fmt.Errorf("failed to update metadata: %w", err)
}
fmt.Println("    Metadata updated successfully\n")

```

```

// 6. Get collection stats
fmt.Println("6. Getting collection statistics...")
stats, err := manager.GetStats(ctx, "tutorial_collection")
if err != nil {
    return fmt.Errorf("failed to get stats: %w", err)
}

fmt.Printf("    Collection Stats:\n")
fmt.Printf("    Total Vectors: %d\n", stats.TotalVectors)
fmt.Printf("    Size: %d bytes\n", stats.TotalSize)
fmt.Printf("    Average Latency: %v\n", stats.AverageLatency)
fmt.Println()

return nil
}

func createSampleVectors() []*memory.VectorData {
    return []*memory.VectorData{
        {
            ID: "doc1",
            Vector: make([]float64, 1536), // In real app, use actual embeddings
            Metadata: map[string]interface{}{
                "title": "Introduction to AI",
                "category": "technology",
                "author": "John Doe",
                "year": 2023,
                "tags": []string{"AI", "machine-learning",
"basics"},
            },
            Collection: "tutorial_collection",
            Timestamp: time.Now(),
        },
        {
            ID: "doc2",
            Vector: make([]float64, 1536),
            Metadata: map[string]interface{}{
                "title": "Modern Web Development",
                "category": "technology",
                "author": "Jane Smith",
                "year": 2023,
                "tags": []string{"web", "development",
"javascript"},
            },
        },
    },
}

```

```

        Collection: "tutorial_collection",
        Timestamp:  time.Now(),
    },
    {
        ID:          "doc3",
        Vector:       make([]float64, 1536),
        Metadata: map[string]interface{}{
            "title":     "Data Science Fundamentals",
            "category":  "science",
            "author":    "Bob Johnson",
            "year":      2022,
            "tags":      []string{"data", "science",
"statistics"},
        },
        Collection: "tutorial_collection",
        Timestamp:  time.Now(),
    },
}

func createQueryVector() []float64 {
    // In real app, use actual embedding of query text
    return make([]float64, 1536)
}

```

Configuration File

```

// helix.json
{
    "$schema": "./schemas/helix.schema.json",
    "version": "1.0.0",
    "environment": "development",
    "debug": true,
    "logging": {
        "level": "info",
        "format": "text",
        "outputs": ["console"]
    },
    "memory": {
        "providers": {
            "chromadb": {
                "type": "chromadb",
                "enabled": true,
                "host": "localhost",

```

```

        "port": 8000,
        "path": "./data/chromadb",
        "timeout": "30s",
        "max_retries": 3,
        "batch_size": 100,
        "compression": true,
        "metric": "cosine",
        "dimension": 1536
    }
},
    "active_provider": "chromadb"
}
}

```

Running the Example

Start ChromaDB

```
docker run -d --name chromadb -p 8000:8000 chromadb/chroma:latest
```

Run the example

```
go run examples/basic/main.go
```

Expected output

```
=== HelixCode Basic Memory Operations ===
```

1. Creating collection...
Collection created successfully
2. Storing sample vectors...
Stored 3 vectors successfully
3. Retrieving vectors...
Retrieved 3 vectors:
 1. ID: doc1, Title: Introduction to AI
 2. ID: doc2, Title: Modern Web Development
 3. ID: doc3, Title: Data Science Fundamentals
4. Performing similarity search...
Found 2 similar documents:
 1. ID: doc1, Score: 0.8923
Title: Introduction to AI
Category: technology
 2. ID: doc2, Score: 0.7845
Title: Modern Web Development

Category: technology

5. Updating vector metadata...

Metadata updated successfully

6. Getting collection statistics...

Collection Stats:

Total Vectors: 3

Size: 48672 bytes

Average Latency: 25ms

TUTORIAL 2: MULTI-PROVIDER SYSTEM

LEARNING OBJECTIVES

- Set up multiple vector providers
- Implement provider switching
- Use load balancing
- Handle fallback scenarios

Complete Code Example

```
// examples/multi_provider/main.go
package main

import (
    "context"
    "fmt"
    "log"
    "time"

    "dev.helix.code/internal/config"
    "dev.helix.code/internal/logging"
    "dev.helix.code/internal/memory"
    "dev.helix.code/internal/memory/providers"
)

func main() {
    ctx := context.Background()
    logger := logging.NewLogger("multi-provider-example")

    // Load configuration with multiple providers
    cfg, err := config.LoadFromFile("helix-multi.json")
```

```

    if err != nil {
        log.Fatal("Failed to load config:", err)
    }

    // Create provider manager
    providerManager := providers.NewVectorProviderManager(
        cfg.Memory.VectorProviders,
        logger,
    )

    // Initialize all providers
    if err := providerManager.Initialize(ctx); err != nil {
        log.Fatal("Failed to initialize providers:", err)
    }
    defer providerManager.Shutdown(ctx)

    // Create memory manager
    memoryManager := memory.NewManager(providerManager, logger)
    if err := memoryManager.Initialize(ctx, cfg); err != nil {
        log.Fatal("Failed to initialize memory:", err)
    }
    defer memoryManager.Shutdown(ctx)

    // Run multi-provider operations
    if err := runMultiProviderOperations(ctx, memoryManager,
        providerManager); err != nil {
        log.Fatal("Multi-provider operations failed:", err)
    }
}

func runMultiProviderOperations(ctx context.Context, manager
    *memory.Manager, pm *providers.VectorProviderManager)
    error {
    fmt.Println("=== Multi-Provider Memory Operations ===\n")

    // 1. Show provider status
    fmt.Println("1. Checking provider status...")
    showProviderStatus(ctx, pm)

    // 2. Test different providers
    fmt.Println("2. Testing different providers...")
    if err := testProviders(ctx, manager, pm); err != nil {
        return fmt.Errorf("failed to test providers: %w", err)
    }
}

```

```

// 3. Test provider switching
fmt.Println("3. Testing provider switching...")
if err := testProviderSwitching(ctx, manager, pm); err !=
    nil {
    return fmt.Errorf("failed to test provider switching:
        %w", err)
}

// 4. Test fallback scenarios
fmt.Println("4. Testing fallback scenarios...")
if err := testFallbackScenarios(ctx, manager, pm); err !=
    nil {
    return fmt.Errorf("failed to test fallback scenarios:
        %w", err)
}

// 5. Performance comparison
fmt.Println("5. Performance comparison...")
if err := comparePerformance(ctx, manager, pm); err != nil {
    return fmt.Errorf("failed to compare performance: %w",
        err)
}

return nil
}

func showProviderStatus(ctx context.Context, pm
    *providers.VectorProviderManager) {
    providers := pm.ListProviders()

    fmt.Printf("    Available Providers (%d):\n", len(providers))
    for name, info := range providers {
        status := "    Unhealthy"
        if info.IsHealthy {
            status = "    Healthy"
        }

        cloudStatus := "    Local"
        if info.IsCloud {
            cloudStatus = "    Cloud"
        }

        activeStatus := ""
        if info.IsActive {

```

```

        activeStatus = " (ACTIVE)"
    }

    fmt.Printf("  %s%s - %s - %s%s\n",
        name, cloudStatus, status,
        info.Type, activeStatus)
}
fmt.Println()
}

func testProviders(ctx context.Context, manager *memory.Manager,
    pm *providers.VectorProviderManager) error {
    // Create test vectors
    vectors := createTestVectors("multi_provider_test")

    // Test each provider
    providers := pm.ListProviders()
    for name := range providers {
        fmt.Printf("Testing provider: %s\n", name)

        // Switch to provider
        if err := pm.SetActiveProvider(ctx, name); err != nil {
            fmt.Printf("    Failed to switch: %v\n", err)
            continue
        }

        // Create collection
        collectionName := fmt.Sprintf("%s_test_collection", name)
        if err := manager.CreateCollection(ctx, collectionName,
            createTestCollectionConfig()); err != nil {
            fmt.Printf("    Failed to create collection: %v\n",
                err)
            continue
        }

        // Store vectors
        start := time.Now()
        if err := manager.Store(ctx, vectors); err != nil {
            fmt.Printf("    Failed to store: %v\n", err)
            continue
        }
        storeDuration := time.Since(start)

        // Search vectors

```

```

    query := createTestQuery(collectionName)
    start = time.Now()
    _, err := manager.Search(ctx, query)
    searchDuration := time.Since(start)

    if err != nil {
        fmt.Printf("        Failed to search: %v\n", err)
        continue
    }

    fmt.Printf("        Store: %v, Search: %v\n",
        storeDuration, searchDuration)
}

fmt.Println()
return nil
}

func testProviderSwitching(ctx context.Context, manager
    *memory.Manager, pm *providers.VectorProviderManager)
    error {
    fmt.Println("Testing provider switching...")

    // Store vectors in first provider
    vectors := createTestVectors("switch_test")
    originalProvider := pm.GetActiveProvider()

    if err := manager.Store(ctx, vectors); err != nil {
        return fmt.Errorf("failed to store vectors: %w", err)
    }

    // Switch to different provider
    providers := pm.ListProviders()
    for name := range providers {
        if name == originalProvider {
            continue
        }

        fmt.Printf("Switching to: %s\n", name)
        if err := pm.SetActiveProvider(ctx, name); err != nil {
            fmt.Printf("Failed to switch: %v\n", err)
            continue
        }
    }
}

```

```

    // Try to retrieve (should fail - different storage)
    ids := []string{vectors[0].ID}
    _, err := manager.Retrieve(ctx, ids)
    if err != nil {
        fmt.Printf("    Expected: vectors not found in new
provider\n")
    }
}

// Switch back
fmt.Printf("Switching back to: %s\n", originalProvider)
if err := pm.SetActiveProvider(ctx, originalProvider); err !=
    nil {
    return fmt.Errorf("failed to switch back: %w", err)
}

// Verify retrieval works
ids := []string{vectors[0].ID}
retrieved, err := manager.Retrieve(ctx, ids)
if err != nil {
    return fmt.Errorf("failed to retrieve after switching
back: %w", err)
}

fmt.Printf("    Retrieved %d vectors after switching
back\n\n", len(retrieved))
return nil
}

func testFallbackScenarios(ctx context.Context, manager
    *memory.Manager, pm *providers.VectorProviderManager)
    error {
    fmt.Println("Testing fallback scenarios...")

    // Configure fallback
    originalProvider := pm.GetActiveProvider()

    // Simulate provider failure
    fmt.Printf("Simulating failure of: %s\n", originalProvider)

    // This would require mocking provider failures
    // For now, we'll just show the concept
    vectors := createTestVectors("fallback_test")

    // Store would normally fail and fallback would kick in

```

```

if err := manager.Store(ctx, vectors); err != nil {
    fmt.Printf("Store failed: %v\n", err)

    // Fallback logic would try other providers
    providers := pm.ListProviders()
    for name := range providers {
        if name == originalProvider {
            continue
        }

        fmt.Printf("Trying fallback to: %s\n", name)
        if err := pm.SetActiveProvider(ctx, name); err ==
nil {
            if fallbackErr := manager.Store(ctx, vectors);
            fallbackErr == nil {
                fmt.Printf("    Fallback successful to:
                %s\n", name)
                break
            }
        }
    }
}

fmt.Println()
return nil
}

func comparePerformance(ctx context.Context, manager
    *memory.Manager, pm *providers.VectorProviderManager)
    error {
    fmt.Println("Performance comparison...")

    vectors := createLargeTestVectorSet("performance_test")

    // Test each provider
    providers := pm.ListProviders()
    results := make(map[string]struct {
        storeTime time.Duration
        searchTime time.Duration
        error      error
    })

    for name := range providers {
        fmt.Printf("Testing performance of: %s\n", name)

```

```

// Switch provider
if err := pm.SetActiveProvider(ctx, name); err != nil {
    results[name] = struct {
        storeTime time.Duration
        searchTime time.Duration
        error      error
    }{error: err}
    continue
}

// Create collection
collectionName := fmt.Sprintf("perf_test_%s", name)
if err := manager.CreateCollection(ctx, collectionName,
createTestCollectionConfig()); err != nil {
    results[name] = struct {
        storeTime time.Duration
        searchTime time.Duration
        error      error
    }{error: err}
    continue
}

// Measure store performance
start := time.Now()
if err := manager.Store(ctx, vectors); err != nil {
    results[name] = struct {
        storeTime time.Duration
        searchTime time.Duration
        error      error
    }{storeTime: time.Since(start), error: err}
    continue
}
storeTime := time.Since(start)

// Measure search performance
query := createTestQuery(collectionName)
start = time.Now()
if _, err := manager.Search(ctx, query); err != nil {
    results[name] = struct {
        storeTime time.Duration
        searchTime time.Duration
        error      error
    }{storeTime: storeTime, searchTime:
time.Since(start), error: err}
}

```



```

        continue
    }
    searchTime := time.Since(start)

    results[name] = struct {
        storeTime time.Duration
        searchTime time.Duration
        error      error
    }{storeTime: storeTime, searchTime: searchTime}

    fmt.Printf("  Store: %v (%.2f vectors/sec)\n",
        storeTime, float64(len(vectors))/storeTime.Seconds())
    fmt.Printf("  Search: %v\n", searchTime)
}

// Print comparison
fmt.Println("\nPerformance Comparison:")
fmt.Printf("%-20s %-15s %-15s %-10s\n", "Provider", "Store
    Time", "Search Time", "Status")
fmt.Println(strings.Repeat("-", 60))

for name, result := range results {
    status := "    Success"
    if result.error != nil {
        status = "    Failed"
    }

    fmt.Printf("%-20s %-15v %-15v %-10s\n",
        name, result.storeTime, result.searchTime, status)
}

return nil
}

func createTestVectors(prefix string) []*memory.VectorData {
    vectors := make([]*memory.VectorData, 10)
    for i := 0; i < 10; i++ {
        vectors[i] = &memory.VectorData{
            ID:      fmt.Sprintf("%s_doc_%d", prefix, i),
            Vector:   make([]float64, 1536), // Mock vector
            Metadata: map[string]interface{}{
                "title":   fmt.Sprintf("Document %d", i),
                "index":   i,
                "prefix":  prefix,
            },
        }
    }
}

```

```

        },
        Collection: fmt.Sprintf("%s_collection", prefix),
        Timestamp: time.Now(),
    }
}
return vectors
}

func createLargeTestVectorSet(prefix string)
[]*memory.VectorData {
vectors := make([]*memory.VectorData, 100)
for i := 0; i < 100; i++ {
    vectors[i] = &memory.VectorData{
        ID:      fmt.Sprintf("%s_large_doc_%d", prefix, i),
        Vector:   make([]float64, 1536), // Mock vector
        Metadata: map[string]interface{}{
            "title":   fmt.Sprintf("Large Document %d", i),
            "index":   i,
            "prefix":  prefix,
        },
        Collection: fmt.Sprintf("%s_large_collection",
prefix),
        Timestamp: time.Now(),
    }
}
return vectors
}

func createTestCollectionConfig() *memory.CollectionConfig {
return &memory.CollectionConfig{
    Name:      "test_collection",
    Dimension: 1536,
    Metric:    "cosine",
    Metadata: map[string]interface{}{
        "description": "Test collection for multi-provider
tutorial",
    },
}
}

func createTestQuery(collection string) *memory.VectorQuery {
return &memory.VectorQuery{
    Vector:     make([]float64, 1536), // Mock query vector
    Collection: collection,
}
}

```

```
    TopK:      5,  
    Threshold: 0.7,  
  }  
}
```

Multi-Provider Configuration

```
// helix-multi.json  
{  
  "$schema": "./schemas/helix.schema.json",  
  "version": "1.0.0",  
  "environment": "development",  
  "debug": true,  
  "logging": {  
    "level": "info",  
    "format": "text",  
    "outputs": ["console"]  
  },  
  "memory": {  
    "providers": {  
      "chromadb": {  
        "type": "chromadb",  
        "enabled": true,  
        "host": "localhost",  
        "port": 8000,  
        "path": "./data/chromadb",  
        "timeout": "30s",  
        "max_retries": 3,  
        "batch_size": 100,  
        "compression": true,  
        "metric": "cosine",  
        "dimension": 1536  
      },  
      "pinecone": {  
        "type": "pinecone",  
        "enabled": false,  
        "api_key": "${PINECONE_API_KEY}",  
        "environment": "us-west1-gcp",  
        "project_id": "test-project",  
        "index_name": "helix-multi-test",  
        "dimension": 1536,  
        "metric": "cosine",  
        "pod_type": "p1.x1",  
        "pods": 1,  
      },  
    },  
  },  
}
```


Complete Code Example

```
// examples/cognee/main.go
package main

import (
    "context"
    "fmt"
    "log"
    "strings"
    "time"

    "dev.helix.code/internal/config"
    "dev.helix.code/internal/logging"
    "dev.helix.code/internal/memory"
    "dev.helix.code/internal/memory/providers"
    "dev.helix.code/internal/memory/cognee_integration"
)

func main() {
    ctx := context.Background()
    logger := logging.NewLogger("cognee-example")

    // Load configuration
    cfg, err := config.LoadFromFile("helix-cognee.json")
    if err != nil {
        log.Fatal("Failed to load config:", err)
    }

    // Create provider manager
    providerManager := providers.NewVectorProviderManager(
        cfg.Memory.VectorProviders,
        logger,
    )

    // Initialize providers
    if err := providerManager.Initialize(ctx); err != nil {
        log.Fatal("Failed to initialize providers:", err)
    }
    defer providerManager.Shutdown(ctx)

    // Create API key manager
    apiKeyManager, _ := config.NewAPIKeyManager(cfg)
```

```

// Create Cognee integration
cogneeIntegration := memory.NewCogneeIntegration(
    providerManager,
    providerManager, // Use same as provider manager for
    simplicity
    apiKeyManager,
)

// Initialize Cognee
if err := cogneeIntegration.Initialize(ctx, cfg.Cognee);
    err != nil {
    log.Fatal("Failed to initialize Cognee:", err)
}
defer cogneeIntegration.Shutdown(ctx)

// Run Cognee operations
if err := runCogneeOperations(ctx, cogneeIntegration); err !=
    nil {
    log.Fatal("Cognee operations failed:", err)
}
}

func runCogneeOperations(ctx context.Context, ci
    *memory.CogneeIntegration) error {
fmt.Println("=== Cognee Integration Operations ===\n")

// 1. Store conversation with Cognee optimization
fmt.Println("1. Storing conversation with Cognee
    optimization...")
if err := storeConversationWithCognee(ctx, ci); err != nil {
    return fmt.Errorf("failed to store conversation: %w",
        err)
}
fmt.Println("    Conversation stored with Cognee
    optimization\n")

// 2. Retrieve context with Cognee
fmt.Println("2. Retrieving context with Cognee...")
if err := retrieveContextWithCognee(ctx, ci); err != nil {
    return fmt.Errorf("failed to retrieve context: %w", err)
}
fmt.Println("    Context retrieved successfully\n")

// 3. Test host-aware optimization
fmt.Println("3. Testing host-aware optimization...")

```

```

    if err := testHostAwareOptimization(ctx, ci); err != nil {
        return fmt.Errorf("failed to test host-aware
            optimization: %w", err)
    }
    fmt.Println("    Host-aware optimization tested\n")

    // 4. Test research-based optimization
    fmt.Println("4. Testing research-based optimization...")
    if err := testResearchBasedOptimization(ctx, ci); err != nil
    {
        return fmt.Errorf("failed to test research-based
            optimization: %w", err)
    }
    fmt.Println("    Research-based optimization tested\n")

    // 5. Build conversation context
    fmt.Println("5. Building conversation context...")
    if err := buildConversationContext(ctx, ci); err != nil {
        return fmt.Errorf("failed to build conversation context:
            %w", err)
    }
    fmt.Println("    Conversation context built successfully\n")

    return nil
}

func storeConversationWithCognee(ctx context.Context, ci
    *memory.CogneeIntegration) error {
    // Create conversation data
    conversation := &memory.ConversationData{
        ID: "conv_001",
        Messages: createConversationMessages(),
        Metadata: map[string]interface{}{
            "title": "AI Assistant Conversation",
            "participants": []string{"user", "assistant"},
            "topic": "artificial intelligence",
            "created_at": time.Now(),
        },
    },
}

    // Store with Cognee optimization
    start := time.Now()
    err := ci.StoreMemory(ctx, &memory.MemoryData{
        ID: conversation.ID,
        Type: memory.MemoryTypeConversation,
    })

```

```

        Content: formatConversation(conversation),
        Source:   "cognee_tutorial",
        Metadata: conversation.Metadata,
        Timestamp: time.Now(),
    })
    duration := time.Since(start)

    if err != nil {
        return fmt.Errorf("failed to store conversation: %w",
            err)
    }

    fmt.Printf("    Stored %d messages in %v\n",
        len(conversation.Messages), duration)
    return nil
}

func retrieveContextWithCognee(ctx context.Context, ci
    *memory.CogneeIntegration) error {
    // Get context for AI assistant
    context, err := ci.GetContext(ctx, "openai", "gpt-4",
        "session_001")
    if err != nil {
        return fmt.Errorf("failed to get context: %w", err)
    }

    fmt.Printf("    Context Size: %d characters\n",
        len(context.Context))
    fmt.Printf("    Memory Entries: %d\n", len(context.Memory))
    fmt.Printf("    Last Updated: %v\n", context.LastUpdated)

    // Display context summary
    if len(context.Memory) > 0 {
        fmt.Printf("    Recent Memory:\n")
        for i, mem := range context.Memory {
            if i >= 3 { // Show only first 3
                fmt.Printf("        ... and %d more\n",
                    len(context.Memory)-i)
                break
            }
            fmt.Printf("        - %s: %s\n", mem.Type,
                mem.Content[:50] + "...")
        }
    }
}

```



```

    return nil
}

func testHostAwareOptimization(ctx context.Context, ci
    *memory.CogneeIntegration) error {
    // Get system information
    sysInfo, err := ci.GetSystemInfo(ctx)
    if err != nil {
        return fmt.Errorf("failed to get system info: %w", err)
    }

    fmt.Printf("  System Info:\n")
    fmt.Printf("    CPU Cores: %d\n", sysInfo.CPUCores)
    fmt.Printf("    Memory: %d MB\n", sysInfo.TotalMemory/1024/
        1024)
    fmt.Printf("    Disk Space: %d GB\n", sysInfo.DiskSpace/1024/
        1024/1024)
    fmt.Printf("    GPU Available: %t\n", sysInfo.GPUAvailable)

    // Test optimization based on system
    testVectors := createOptimizationTestVectors()

    fmt.Printf("  Testing optimization for %d vectors...\n",
        len(testVectors))
    start := time.Now()
    err = ci.StoreMemory(ctx, &memory.MemoryData{
        ID:      "optimization_test",
        Type:     memory.MemoryTypeKnowledge,
        Content:  "Optimization test data",
        Source:   "cognee_tutorial",
        Metadata: map[string]interface{}{
            "test_type": "host_aware_optimization",
            "vector_count": len(testVectors),
            "timestamp": time.Now(),
        },
        Timestamp: time.Now(),
    })
    duration := time.Since(start)

    if err != nil {
        return fmt.Errorf("failed to store optimization test:
            %w", err)
    }

    fmt.Printf("  Optimized storage completed in %v\n", duration)

```

```

    return nil
}

func testResearchBasedOptimization(ctx context.Context, ci
    *memory.CogneeIntegration) error {
    // Get research-based recommendations
    recommendations, err :=
        ci.GetOptimizationRecommendations(ctx)
    if err != nil {
        return fmt.Errorf("failed to get recommendations: %w",
            err)
    }

    fmt.Printf("  Research-Based Recommendations:\n")
    for _, rec := range recommendations {
        fmt.Printf("    - %s: %s\n", rec.Category,
            rec.Recommendation)
        fmt.Printf("      Confidence: %.2f\n", rec.Confidence)
        fmt.Printf("      Research: %s\n", rec.ResearchPaper)
    }

    // Apply recommendations
    fmt.Printf("  Applying optimization recommendations...\n")
    err = ci.ApplyOptimizations(ctx, recommendations)
    if err != nil {
        return fmt.Errorf("failed to apply optimizations: %w",
            err)
    }

    fmt.Printf("    Optimizations applied successfully\n")
    return nil
}

func buildConversationContext(ctx context.Context, ci
    *memory.CogneeIntegration) error {
    // Create conversation manager
    convManager := memory.NewConversationManager(ci,
        logging.NewLogger("conv_manager"))

    // Add messages to conversation
    messages := createConversationMessages()
    sessionID := "session_001"

    fmt.Printf("  Building conversation context for session %s...
        \n", sessionID)

```

```

for _, msg := range messages {
    if err := convManager.AddMessage(ctx, sessionID, msg);
    err != nil {
        return fmt.Errorf("failed to add message: %w", err)
    }
}

// Get conversation summary
summary, err := convManager.GetSummary(ctx, sessionID)
if err != nil {
    return fmt.Errorf("failed to get summary: %w", err)
}

fmt.Printf("  Conversation Summary:\n")
fmt.Printf("    Total Messages: %d\n", summary.TotalMessages)
fmt.Printf("    Duration: %v\n", summary.Duration)
fmt.Printf("    Topics: %v\n", summary.Topics)
fmt.Printf("    Sentiment: %s\n", summary.Sentiment)

// Get context window
contextWindow, err := convManager.GetContextWindow(ctx,
    sessionID, 5)
if err != nil {
    return fmt.Errorf("failed to get context window: %w",
    err)
}

fmt.Printf("  Context Window (last 5 messages):\n")
for i, msg := range contextWindow {
    fmt.Printf("    %d. %s: %s\n", i+1, msg.Role,
    msg.Content[:50] + "...")
}

return nil
}

func createConversationMessages() []*memory.ConversationMessage {
    return []*memory.ConversationMessage{
        {
            ID:          "msg_001",
            Role:        "user",
            Content:     "Hello! I'm interested in learning about
artificial intelligence.",
            Timestamp: time.Now().Add(-10 * time.Minute),
            Metadata:  map[string]interface{}{}
        }
    }
}

```

```

        "source": "web_chat",
        "language": "en",
    },
},
{
    ID:          "msg_002",
    Role:        "assistant",
    Content:     "Hello! I'd be happy to help you learn
about AI. What specific aspect of artificial intelligence
interests you most?",
    Timestamp:   time.Now().Add(-9 * time.Minute),
    Metadata:    map[string]interface{}{
        "model": "gpt-4",
        "tokens": 25,
    },
},
{
    ID:          "msg_003",
    Role:        "user",
    Content:     "I'm particularly interested in machine
learning and neural networks. Can you explain the
basics?",
    Timestamp:   time.Now().Add(-8 * time.Minute),
    Metadata:    map[string]interface{}{
        "source": "web_chat",
        "language": "en",
    },
},
{
    ID:          "msg_004",
    Role:        "assistant",
    Content:     "Machine learning is a subset of AI that
focuses on systems that can learn from data. Neural
networks are computing systems inspired by biological
neural networks. Let me break this down for you...",
    Timestamp:   time.Now().Add(-7 * time.Minute),
    Metadata:    map[string]interface{}{
        "model": "gpt-4",
        "tokens": 45,
    },
},
},
}
}

```

```

func formatConversation(conv *memory.ConversationData) string {

```

```

var builder strings.Builder

for _, msg := range conv.Messages {
    builder.WriteString(fmt.Sprintf("[%s] %s: %s\n",
        msg.Timestamp.Format("15:04:05"),
        msg.Role,
        msg.Content))
}

return builder.String()
}

func createOptimizationTestVectors() []float64 {
    // Create a larger test vector set for optimization
    return make([]float64, 1536*100) // Mock 100 vectors
    concatenated
}

```

TUTORIAL 4: BUILDING AI CHATBOT WITH MEMORY

LEARNING OBJECTIVES

- Build complete AI chatbot with memory
- Implement conversation context
- Use vector search for knowledge retrieval
- Add personality and character traits

Complete Code Example

```

// examples/chatbot/main.go
package main

import (
    "bufio"
    "context"
    "fmt"
    "log"
    "os"
    "strings"
    "time"

    "dev.helix.code/internal/config"

```

```

    "dev.helix.code/internal/logging"
    "dev.helix.code/internal/memory"
    "dev.helix.code/internal/memory/providers"
    "dev.helix.code/internal/providers/llm"
)

type Chatbot struct {
    memoryManager      *memory.Manager
    conversationMgr     *memory.ConversationManager
    llmProvider         providers.LLMProvider
    logger              logging.Logger
    personality         *Personality
}

type Personality struct {
    Name      string
    Traits    map[string]string
    Responses map[string][]string
    Knowledge []string
}

func main() {
    ctx := context.Background()

    // Initialize chatbot
    chatbot, err := NewChatbot(ctx)
    if err != nil {
        log.Fatal("Failed to initialize chatbot:", err)
    }
    defer chatbot.Shutdown(ctx)

    // Start interactive chat
    chatbot.StartInteractiveChat(ctx)
}

func NewChatbot(ctx context.Context) (*Chatbot, error) {
    logger := logging.NewLogger("helixbot")

    // Load configuration
    cfg, err := config.LoadFromFile("helix-chatbot.json")
    if err != nil {
        return nil, fmt.Errorf("failed to load config: %w", err)
    }

```

```

// Initialize providers
providerManager := providers.NewVectorProviderManager(
    cfg.Memory.VectorProviders,
    logger,
)

if err := providerManager.Initialize(ctx); err != nil {
    return nil, fmt.Errorf("failed to initialize providers:
        %w", err)
}

// Initialize memory
memoryManager := memory.NewManager(providerManager, logger)
if err := memoryManager.Initialize(ctx, cfg); err != nil {
    return nil, fmt.Errorf("failed to initialize memory:
        %w", err)
}

// Initialize conversation manager
conversationMgr :=
    memory.NewConversationManager(memoryManager, logger)

// Initialize LLM provider
llmProvider, err :=
    providers.NewLLMProvider(cfg.Providers.OpenAI)
if err != nil {
    return nil, fmt.Errorf("failed to initialize LLM
        provider: %w", err)
}

if err := llmProvider.Initialize(ctx,
    cfg.Providers.OpenAI.Configuration); err != nil {
    return nil, fmt.Errorf("failed to initialize LLM: %w",
        err)
}

// Create personality
personality := createPersonality()

return &Chatbot{
    memoryManager:    memoryManager,
    conversationMgr:  conversationMgr,
    llmProvider:      llmProvider,
    logger:           logger,
    personality:      personality,
}

```

```

    }, nil
}

func (c *Chatbot) StartInteractiveChat(ctx context.Context) {
    fmt.Printf("    %s is online! Type 'quit' to exit.\n\n",
        c.personality.Name)

    scanner := bufio.NewScanner(os.Stdin)
    sessionID := fmt.Sprintf("session_%d", time.Now().Unix())

    for {
        fmt.Printf("You: ")
        if !scanner.Scan() {
            break
        }

        userMessage := strings.TrimSpace(scanner.Text())
        if userMessage == "quit" || userMessage == "exit" {
            fmt.Printf("%s: Goodbye! It was nice talking to you.\n",
                c.personality.Name)
            break
        }

        if userMessage == "" {
            continue
        }

        // Process message
        response, err := c.ProcessMessage(ctx, sessionID,
            userMessage)
        if err != nil {
            c.logger.Error("Failed to process message", "error",
                err)
            fmt.Printf("%s: I'm having trouble processing that. Can you try again?\n", c.personality.Name)
            continue
        }

        fmt.Printf("%s: %s\n\n", c.personality.Name, response)
    }
}

func (c *Chatbot) ProcessMessage(ctx context.Context, sessionID,
    userMessage string) (string, error) {
    // Add user message to conversation

```



```

userMsg := &memory.ConversationMessage{
    ID:          fmt.Sprintf("user_%d", time.Now().UnixNano()),
    Role:        "user",
    Content:     userMessage,
    Timestamp:   time.Now(),
    Metadata:    map[string]interface{}{
        "session_id": sessionID,
    },
}

if err := c.conversationMgr.AddMessage(ctx, sessionID,
    userMsg); err != nil {
    return "", fmt.Errorf("failed to add user message: %w",
        err)
}

// Retrieve relevant knowledge
knowledge, err := c.RetrieveRelevantKnowledge(ctx,
    userMessage)
if err != nil {
    c.logger.Warn("Failed to retrieve knowledge", "error",
        err)
    knowledge = ""
}

// Get conversation context
contextWindow, err :=
    c.conversationMgr.GetContextWindow(ctx, sessionID, 5)
if err != nil {
    c.logger.Warn("Failed to get context window", "error",
        err)
    contextWindow = []*memory.ConversationMessage{}
}

// Build prompt
prompt := c.BuildPrompt(userMessage, contextWindow,
    knowledge)

// Generate response
response, err := c.GenerateResponse(ctx, prompt)
if err != nil {
    return "", fmt.Errorf("failed to generate response: %w",
        err)
}

```

```

// Add assistant message to conversation
assistantMsg := &memory.ConversationMessage{
    ID:          fmt.Sprintf("assistant_%d",
        time.Now().UnixNano()),
    Role:        "assistant",
    Content:     response,
    Timestamp:   time.Now(),
    Metadata:    map[string]interface{}{
        "session_id": sessionID,
        "prompt":     prompt,
        "knowledge":   knowledge,
    },
}

if err := c.conversationMgr.AddMessage(ctx, sessionID,
    assistantMsg); err != nil {
    c.logger.Warn("Failed to add assistant message",
        "error", err)
}

return response, nil
}

func (c *Chatbot) RetrieveRelevantKnowledge(ctx context.Context,
    query string) (string, error) {
    // Create embedding for query (in real app, use embedding
    // service)
    queryVector := make([]float64, 1536) // Mock embedding

    // Search memory
    searchQuery := &memory.VectorQuery{
        Vector:      queryVector,
        Collection:   "knowledge_base",
        TopK:        3,
        Threshold:    0.7,
        Filters:     map[string]interface{}{
            "type": "knowledge",
        },
    }

    results, err := c.memoryManager.Search(ctx, searchQuery)
    if err != nil {
        return "", fmt.Errorf("failed to search knowledge: %w",
            err)
    }
}

```

```

// Format knowledge
var knowledge strings.Builder
for i, result := range results.Results {
    if i > 0 {
        knowledge.WriteString("\n")
    }
    knowledge.WriteString(fmt.Sprintf("Knowledge %d: %s",
        i+1, result.Metadata["content"]))
}

return knowledge.String(), nil
}

func (c *Chatbot) BuildPrompt(userMessage string, context
    []*memory.ConversationMessage, knowledge string) string {
    var builder strings.Builder

    // System prompt with personality
    builder.WriteString(fmt.Sprintf("You are %s, an AI assistant.
        \n", c.personality.Name))
    builder.WriteString("Personality traits:\n")
    for trait, description := range c.personality.Traits {
        builder.WriteString(fmt.Sprintf("- %s: %s\n", trait,
            description))
    }
    builder.WriteString("\n")

    // Knowledge
    if knowledge != "" {
        builder.WriteString("Relevant knowledge:\n")
        builder.WriteString(knowledge)
        builder.WriteString("\n\n")
    }

    // Conversation context
    if len(context) > 0 {
        builder.WriteString("Recent conversation:\n")
        for _, msg := range context {
            builder.WriteString(fmt.Sprintf("%s: %s\n",
                msg.Role, msg.Content))
        }
        builder.WriteString("\n")
    }
}

```

```

    // Current message
    builder.WriteString(fmt.Sprintf("User message: %s\n\n",
        userMessage))
    builder.WriteString("Please respond in character, being
        helpful and engaging.")

    return builder.String()
}

func (c *Chatbot) GenerateResponse(ctx context.Context, prompt
    string) (string, error) {
    request := &providers.LLMRequest{
        Model: "gpt-4",
        Messages: []providers.LLMMessage{
            {
                Role:    "system",
                Content: "You are a helpful AI assistant.",
            },
            {
                Role:    "user",
                Content: prompt,
            },
        },
        Temperature: 0.7,
        MaxTokens:    500,
        Stream:       false,
    }

    response, err := c.llmProvider.Chat(ctx, request)
    if err != nil {
        return "", fmt.Errorf("failed to generate response: %w",
            err)
    }

    return response.Content, nil
}

func (c *Chatbot) Shutdown(ctx context.Context) {
    if c.memoryManager != nil {
        c.memoryManager.Shutdown(ctx)
    }

    if c.llmProvider != nil {
        c.llmProvider.Shutdown(ctx)
    }
}

```

```

    c.logger.Info("Chatbot shutdown complete")
}

func createPersonality() *Personality {
    return &Personality{
        Name: "Helix",
        Traits: map[string]string{
            "friendly":      "Warm and approachable",
            "knowledgeable": "Well-informed and accurate",
            "curious":        "Asks thoughtful questions",
            "humorous":       "Has a light sense of humor",
            "empathetic":     "Shows understanding and care",
        },
        Responses: map[string][]string{
            "greeting": {
                "Hello! How can I help you today?",
                "Hi there! What's on your mind?",
                "Greetings! What can I assist you with?",
            },
            "farewell": {
                "Goodbye! It was nice talking to you!",
                "See you later! Feel free to come back anytime.",
                "Take care! Have a wonderful day!",
            },
            "thinking": {
                "Let me think about that...",
                "That's an interesting question...",
                "Hmm, let me consider that carefully...",
            },
        },
        Knowledge: []string{
            "I'm designed to be helpful and conversational.",
            "I learn from our conversations to provide better responses.",
            "I have access to a wide range of knowledge.",
            "I respect your privacy and store conversations securely.",
        },
    }
}

```

CONFIGURATION EXAMPLES

Basic Configuration

```
{
  "$schema": "../schemas/helix.schema.json",
  "version": "1.0.0",
  "environment": "development",
  "debug": true,
  "logging": {
    "level": "info",
    "format": "text",
    "outputs": ["console"]
  },
  "memory": {
    "providers": {
      "chromadb": {
        "type": "chromadb",
        "enabled": true,
        "host": "localhost",
        "port": 8000,
        "path": "../data/chromadb",
        "metric": "cosine",
        "dimension": 1536
      }
    },
    "active_provider": "chromadb"
  },
  "cognee": {
    "enabled": true,
    "mode": "local",
    "optimization": {
      "host_aware": true,
      "research_based": true,
      "auto_tune": true
    }
  },
  "providers": {
    "openai": {
      "enabled": true,
      "api_key": "${OPENAI_API_KEY}",
      "models": {
        "default": "gpt-4",
        "embedding": "text-embedding-3-large"
      }
    }
  }
}
```

```

    }
  }
}

```

Production Configuration

```

{
  "$schema": "./schemas/helix.schema.json",
  "version": "1.0.0",
  "environment": "production",
  "debug": false,
  "logging": {
    "level": "info",
    "format": "json",
    "outputs": ["file", "elasticsearch"],
    "file": {
      "path": "./logs/helix.log",
      "max_size": "100MB",
      "max_files": 10,
      "rotate": true
    }
  },
  "server": {
    "ssl": {
      "enabled": true,
      "cert_file": "./certs/server.crt",
      "key_file": "./certs/server.key"
    }
  },
  "memory": {
    "providers": {
      "pinecone": {
        "type": "pinecone",
        "enabled": true,
        "api_key": "${PINECONE_API_KEY}",
        "environment": "us-west1-gcp",
        "index_name": "helix-prod",
        "pod_type": "p1.x2",
        "pods": 2,
        "replicas": 2
      }
    }
  },
  "active_provider": "pinecone",

```

```

        "fallback": {
            "enabled": true,
            "providers": ["pinecone", "chromadb"],
            "retry_count": 3
        }
    },
    "security": {
        "authentication": {
            "enabled": true,
            "method": "jwt"
        },
        "encryption": {
            "enabled": true,
            "algorithm": "AES-256-GCM"
        }
    }
}

```

PERFORMANCE MONITORING

Monitoring Dashboard

```

// examples/monitoring/main.go
package main

import (
    "context"
    "fmt"
    "log"
    "time"

    "dev.helix.code/internal/config"
    "dev.helix.code/internal/logging"
    "dev.helix.code/internal/memory/providers"
)

func main() {
    ctx := context.Background()
    logger := logging.NewLogger("monitoring")

    // Load configuration
    cfg, err := config.LoadFromFile("helix.json")
    if err != nil {

```



```

        log.Fatal("Failed to load config:", err)
    }

    // Create provider manager
    providerManager := providers.NewVectorProviderManager(
        cfg.Memory.VectorProviders,
        logger,
    )

    // Initialize
    if err := providerManager.Initialize(ctx); err != nil {
        log.Fatal("Failed to initialize:", err)
    }
    defer providerManager.Shutdown(ctx)

    // Start monitoring
    startMonitoring(ctx, providerManager, logger)
}

func startMonitoring(ctx context.Context, pm
    *providers.VectorProviderManager, logger logging.Logger)
{
    ticker := time.NewTicker(30 * time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            monitorProviders(ctx, pm, logger)
        }
    }
}

func monitorProviders(ctx context.Context, pm
    *providers.VectorProviderManager, logger logging.Logger)
{
    // Get provider health
    health, err := pm.GetProviderHealth(ctx)
    if err != nil {
        logger.Error("Failed to get provider health", "error",
            err)
        return
    }
}

```

```

// Get provider performance
performance := pm.GetProviderPerformance()

// Display monitoring data
fmt.Printf("\n=== HelixCode Monitoring Dashboard ===\n")
fmt.Printf("Timestamp: %s\n", time.Now().Format("2006-01-02
    15:04:05"))
fmt.Printf("\nProvider Health:\n")

for name, h := range health {
    status := "    Unhealthy"
    if h.IsHealthy {
        status = "    Healthy"
    }

    fmt.Printf("    %s: %s\n", name, status)
    fmt.Printf("        Response Time: %v\n", h.ResponseTime)
    fmt.Printf("        Error Count: %d\n", h.ErrorCount)
    fmt.Printf("        Last Check: %s\n",
        h.LastCheck.Format("15:04:05"))
}

fmt.Printf("\nProvider Performance:\n")
for name, p := range performance {
    fmt.Printf("    %s:\n", name)
    fmt.Printf("        Total Operations: %d\n",
        p.TotalOperations)
    fmt.Printf("        Success Rate: %.2f%%\n",
        (1-p.ErrorRate)*100)
    fmt.Printf("        Average Latency: %v\n", p.AverageLatency)
    fmt.Printf("        Throughput: %.2f ops/sec\n",
        p.Throughput)
}

// Get active provider
active := pm.GetActiveProvider()
fmt.Printf("\nActive Provider: %s\n", active)

// Get provider list
providers := pm.ListProviders()
fmt.Printf("Total Providers: %d\n", len(providers))

// Alert on issues
for name, h := range health {

```

```

    if !h.IsHealthy {
        logger.Error("Provider health issue detected",
            "provider", name,
            "error", h.ErrorMessage,
            "error_count", h.ErrorCount)
    }

    if h.ResponseTime > 1*time.Second {
        logger.Warn("High response time detected",
            "provider", name,
            "response_time", h.ResponseTime)
    }
}

for name, p := range performance {
    if p.ErrorRate > 0.05 { // 5% error rate threshold
        logger.Error("High error rate detected",
            "provider", name,
            "error_rate", p.ErrorRate)
    }

    if p.AverageLatency > 500*time.Millisecond {
        logger.Warn("High latency detected",
            "provider", name,
            "latency", p.AverageLatency)
    }
}
}
}

```

DEPLOYMENT EXAMPLES

Docker Deployment

```

# Dockerfile
FROM golang:1.19-alpine AS builder

WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 go build -o helixcode ./cmd/server/

```

```
FROM alpine:latest
RUN apk --no-cache add ca-certificates tzdata
WORKDIR /root/

COPY --from=builder /app/helixcode .
COPY --from=builder /app/configs ./configs

EXPOSE 8080
CMD ["/helixcode"]
```

Kubernetes Deployment

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helixcode
  labels:
    app: helixcode
spec:
  replicas: 3
  selector:
    matchLabels:
      app: helixcode
  template:
    metadata:
      labels:
        app: helixcode
    spec:
      containers:
        - name: helixcode
          image: helixcode:latest
          ports:
            - containerPort: 8080
          env:
            - name: HELIX_ENVIRONMENT
              value: "production"
            - name: OPENAI_API_KEY
              valueFrom:
                secretKeyRef:
                  name: helix-secrets
                  key: openai-api-key
            - name: PINECONE_API_KEY
              valueFrom:
```

```

        secretKeyRef:
          name: helix-secrets
          key: pinecone-api-key
resources:
  requests:
    memory: "512Mi"
    cpu: "250m"
  limits:
    memory: "1Gi"
    cpu: "500m"
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: helixcode-service
spec:
  selector:
    app: helixcode
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: LoadBalancer

```

NEXT STEPS

Advanced Topics

- Custom memory providers
- Advanced optimization techniques
- Multi-tenant architectures

- Real-time synchronization
- Disaster recovery

Additional Resources

- [API Reference](#)
- [Configuration Guide](#)
- [Performance Optimization](#)
- [Security Best Practices](#)

This practical guide is continuously updated with new examples and tutorials. Check for the latest version at docs.helixcode.ai.